

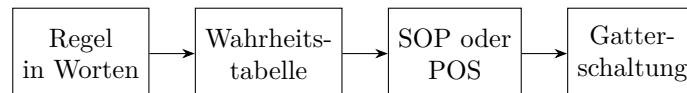
01 / EINORDNEN

Von einer Regel zur Schaltung

Ziel für Tag 7: Du kannst aus einer vollständigen Wahrheitstabelle zwei gleichwertige Gleichungen erzeugen: die kanonische SOP und die kanonische POS. Anschließend überträgst du die Schritte in ein kleines Python-Tool.

Warum machen wir das?

Stell dir drei Sensoren vor. Eine Warnlampe soll leuchten, wenn mindestens zwei Sensoren auslösen. Die Beschreibung ist verständlich, aber zum Bauen oder Programmieren brauchst du eine eindeutige Vorschrift für jede Eingangskombination.



Eine **kombinatorische** Schaltung bestimmt ihren stabilen Ausgang aus den aktuellen Eingängen. Sie braucht dafür keinen gespeicherten früheren Zustand. Die physikalische Verzögerung aus Tag 6 bleibt trotzdem vorhanden. Heute beschreiben wir zunächst die stabilen Logikwerte.

Die Zeichen bedeuten Logik

Schreibweise	Bedeutung	Beispiel
$\bar{A}$ oder $!A$	NICHT A	$\bar{0} = 1$
$A \cdot B$, AB oder $A * B$	A UND B	$1 \cdot 0 = 0$
$A + B$	A ODER B	$1 + 1 = 1$

Achtung: Das Pluszeichen ist hier kein gewöhnliches Addieren. Wir rechnen mit Logikwerten 0 und 1. Reihenfolge: Klammern, NICHT, UND, ODER. Deshalb bedeutet $A + BC$ dasselbe wie $A + (B \cdot C)$.

Vier Wörter, die wir brauchen

Literal: Eine Variable oder ihre Negation, zum Beispiel A oder $\bar{A}$.

Produkt: Mit UND verbundene Literale, etwa $A\bar{B}$.

Summe: Mit ODER verbundene Literale, etwa $A + \bar{B}$.

Kanonisch: Eine festgelegte Standardform. Jeder einzelne Minterm bzw. Maxterm enthält jede Eingangsvariable genau einmal.

Eine Wahrheitstabelle kann mehrere gleichwertige Gleichungen haben. Kanonisch bedeutet eindeutig aufgebaut, nicht automatisch möglichst kurz.

02 / EINE ZEILE ERKENNEN

Der Minterm: genau hier eine 1

Ein **Minterm** ist eine UND-Verknüpfung aller Eingangsvariablen. Er soll für genau eine Tabellenzeile 1 ergeben. Bei allen anderen Zeilen ergibt er 0.

Schritt für Schritt: die Zeile A=1, B=0, C=1

Wir suchen einen Ausdruck, der bei dieser Kombination 1 ergibt. Bei einem UND müssen dafür alle drei Teile 1 sein.

Variable	Wert in der Zeile	Gewähltes Literal	Wert des Literals
A	1	A	1
B	0	$\overline{B}$	1
C	1	C	1

Die drei Literale werden mit UND verbunden:

$$m_5 = \overline{A}BC$$

Die tiefgestellte 5 ist die Zeilennummer: $101_2 = 4 + 1 = 5$. Sie ist kein Exponent und keine Anzahl von Variablen.

Probe an der passenden Zeile

$$m_5(1, 0, 1) = 1 \cdot \overline{0} \cdot 1 = 1 \cdot 1 \cdot 1 = 1$$

Probe an einer anderen Zeile

Ändere nur B von 0 auf 1:

$$m_5(1, 1, 1) = 1 \cdot \overline{1} \cdot 1 = 1 \cdot 0 \cdot 1 = 0$$

Sobald irgendein Eingang von der gesuchten Zeile abweicht, wird das zugehörige Literal 0. Damit wird das gesamte UND 0. Der Ausdruck erkennt genau die gesuchte Kombination.

Minterm-Regel: Tabellenbit 0 → Variable negieren. Tabellenbit 1 → Variable unverändert lassen. Danach alles mit UND verbinden.

Noch zwei Beispiele

Für $ABC = 000$ lautet der Minterm $\overline{A}\overline{B}\overline{C}$. Jedes der drei negierten Literale wird bei 000 zu 1.

Für $ABC = 110$ lautet er $AB\overline{C}$.

AB ist bei einer Funktion mit den Eingängen A, B und C **kein vollständiger Minterm**: C fehlt. Der Ausdruck ist sowohl bei 110 als auch bei 111 gleich 1.

03 / DIE EINERZEILEN SAMMELN

SOP: UND-Terme mit ODER verbinden

SOP bedeutet *Sum of Products*, also ODER-Verknüpfung von UND-Termen. In der **kanonischen SOP** verwenden wir die Minterme aller Zeilen mit Ausgang $Y = 1$.

Durchgerechnetes Beispiel mit zwei Eingängen

Die folgende Tabelle ist die Vorgabe. Die Werte für Y werden nicht aus A und B addiert, sondern beschreiben die gewünschte Funktion.

Index	A	B	Y	Minterm der Zeile
0	0	0	1	$m_0 = \overline{A}\overline{B}$
1	0	1	1	$m_1 = \overline{A}B$
2	1	0	0	$m_2 = A\overline{B}$
3	1	1	1	$m_3 = AB$

Schritt 1: Wähle die Zeilen 0, 1 und 3, denn dort steht $Y = 1$.

Schritt 2: Schreibe für jede ausgewählte Zeile ihren Minterm.

Schritt 3: Verbinde die Minterme mit ODER:

$$Y = \overline{A}\overline{B} + \overline{A}B + AB$$

Warum ODER? Es reicht, wenn **eine** der erlaubten Eingangskombinationen erkannt wird. Die Minterme für die Nullzeilen werden nicht aufgenommen.

Einsetzen statt nur auf das Ergebnis vertrauen

Für $A = 0, B = 1$:

$$Y = (1 \cdot 0) + (1 \cdot 1) + (0 \cdot 1) = 0 + 1 + 0 = 1$$

Für $A = 1, B = 0$:

$$Y = (0 \cdot 1) + (0 \cdot 0) + (1 \cdot 0) = 0 + 0 + 0 = 0$$

Die kurze Schreibweise

$$Y(A, B) = \Sigma m(0, 1, 3)$$

Das bedeutet: Verbinde die Minterme 0, 1 und 3 mit ODER. Σ ist hier eine Abkürzung für die Auswahl der Einerzeilen. Die Variablenreihenfolge ist A, B; A ist das höchstwertige Bit.

SOP-Rezept: Einerzeilen auswählen → je Zeile einen Minterm bilden → diese mit ODER verbinden. Nicht alle Tabellenzeilen übernehmen!

04 / DIE NULLZEILEN AUSSCHLIESSEN

POS: ODER-Klammern mit UND verbinden

POS bedeutet *Product of Sums*. In der **kanonischen POS** verwenden wir die Maxterme aller Zeilen mit $Y = 0$. Ein **Maxterm** ist eine ODER-Klammer mit allen Eingangsvariablen. Er wird für genau eine Zeile 0. Dafür müssen alle Literale der Klammer 0 sein.

Maxterm für A=1, B=0, C=1

Variable	Tabellenbit	Gewähltes Literal	Wert des Literals
A	1	$\overline{A}$	0
B	0	B	0
C	1	$\overline{C}$	0

$$M_5 = (\overline{A} + B + \overline{C}), \quad M_5(1, 0, 1) = 0 + 0 + 0 = 0$$

Ändert sich irgendein Eingang, wird mindestens ein Literal 1 und damit die ganze ODER-Klammer 1.

Maxterm-Regel: Tabellenbit 0 $\rightarrow$ Variable unverändert. Tabellenbit 1 $\rightarrow$ Variable negieren.
Danach mit ODER verbinden.

Zurück zum Beispiel von Seite 3

Dort ist ausschließlich Zeile 2, also $AB = 10$, eine Nullzeile. Ihr Maxterm ist $(\overline{A} + B)$. Deshalb:

$$Y = (\overline{A} + B) = \Pi M(2)$$

Bei mehreren Nullzeilen werden die Klammern mit UND verbunden. Wird eine dieser Klammern 0, wird das Gesamtergebnis 0. Bei allen übrigen Zeilen sind alle ausgewählten Klammern 1.

Warum POS dieselbe Funktion Y ergibt

Bei unserem Beispiel ist $\overline{Y}$ nur in Zeile 2 gleich 1. Also:

$$\overline{Y} = A\overline{B} \Rightarrow Y = \overline{A\overline{B}} = \overline{A} + \overline{\overline{B}} = \overline{A} + B$$

Das ist De Morgan. **Die fertige POS beschreibt Y, nicht $\overline{Y}$.** Wir benutzen die Nullzeilen, um festzulegen, wann Y ausgeschaltet sein muss.

05 / ORDNUNG UND VEREINFACHUNG

Was bedeutet die Zeilennummer?

Für drei Eingänge verwenden wir immer die Reihenfolge A, B, C. A zählt als höchstwertiges Bit, C wechselt am schnellsten. Der Index ist $i = 4A + 2B + C$.

Index	A	B	C	Binärmuster
0	0	0	0	000
1	0	0	1	001
2	0	1	0	010
3	0	1	1	011
4	1	0	0	100
5	1	0	1	101
6	1	1	0	110
7	1	1	1	111

Drei Eingänge ergeben $2^3 = 8$ Zeilen. Allgemein ergeben n Eingänge 2^n Zeilen. Ein Index von 5 meint die Kombination 101; er meint nicht den Ausgangswert 5.

SOP und POS im direkten Vergleich

	Kanonische SOP	Kanonische POS
Ausgewählte Zeilen	$Y = 1$	$Y = 0$
Tabellenbit 0	negiertes Literal	normales Literal
Tabellenbit 1	normales Literal	negiertes Literal
Innerhalb eines Terms	UND	ODER
Zwischen den Termen	ODER	UND

Kanonisch und vereinfacht sind nicht dasselbe

Die SOP unseres Beispiels lässt sich kürzen:

$$\begin{aligned}
 Y &= \overline{A}\overline{B} + \overline{A}B + AB \\
 &= \overline{A}(\overline{B} + B) + AB && \text{ausklammern} \\
 &= \overline{A} + AB && \text{weil } \overline{B} + B = 1 \\
 &= (\overline{A} + A)(\overline{A} + B) && \text{Distributivgesetz} \\
 &= \overline{A} + B && \text{weil } \overline{A} + A = 1
 \end{aligned}$$

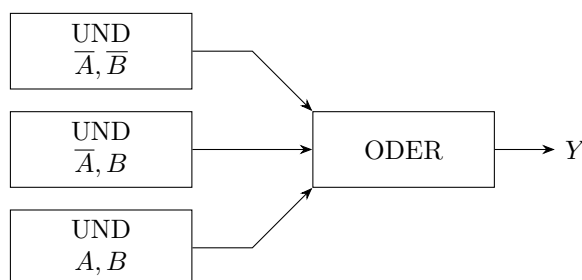
Die verkürzte Gleichung ist gleichwertig. Als SOP über zwei Variablen ist sie aber nicht mehr kanonisch: Die einzelnen Terme enthalten nicht beide Variablen. Als POS ist $(\overline{A} + B)$ hier weiterhin kanonisch.

06 / VON DER GLEICHUNG ZUM AUFBAU

Gatter und vollständige Kontrolle

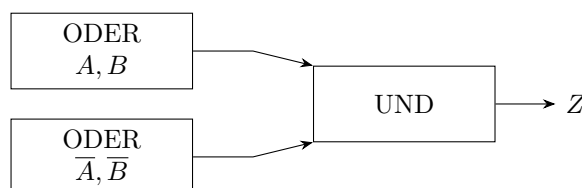
SOP: erst UND, dann ODER

Für $Y = \overline{A}\overline{B} + \overline{A}B + AB$ bildet jedes UND einen Minterm. Ein ODER sammelt die drei Ergebnisse. Die Rechtecke unten stehen für die genannten Gatter; nötige Negationen werden durch NOT-Gatter bereitgestellt.



POS: erst ODER, dann UND

Ein Beispiel mit zwei Maxtermen ist $Z = (A + B)(\overline{A} + \overline{B})$. Zwei ODER-Gatter bilden die Klammern, ein UND verbindet sie:



Wann sind zwei Gleichungen gleichwertig?

Wenn sie bei **jeder** möglichen Eingangskombination denselben Ausgang liefern. Ein passendes Beispiel beweist keine Gleichwertigkeit. Ein einziges Gegenbeispiel reicht dagegen aus, um eine behauptete Gleichwertigkeit zu widerlegen.

Die zwei Prüf-Funktionen aus deinem Lernplan

Majority3: Der Ausgang ist 1, wenn mindestens zwei der drei Eingänge 1 sind. Auch drei Einsen zählen dazu.

XOR3: Der Ausgang ist 1, wenn die Anzahl der Einsen ungerade ist, also bei einer oder drei Einsen. Bei drei Eingängen bedeutet XOR nicht einfach „genau einer ist 1“.

07 / PYTHON ALS ÜBERSETZER

Aus einer Liste wird eine Gleichung

Das Tool baut Text. Es vereinfacht die Gleichung zunächst nicht und simuliert keine Transistoren. Die Eingabe besteht aus Variablennamen und einer Liste der Ausgangswerte in aufsteigender Zeilenreihenfolge.

```
names = ["A", "B"]
outputs = [1, 1, 0, 1]
# Index:  0  1  2  3
# A,B:   00 01 10 11
```

1. Den Index in Tabellenbits umwandeln

Du kennst das bereits von Tag 4:

```
index = 5
n = 3
bits = bin(index)[2:].zfill(n)
# bin(5) -> "0b101"; [2:] -> "101"; zfill(3) -> "101"
```

Bei Index 1 und drei Variablen entsteht "001". Führende Nullen sind hier notwendig, damit jedes Bit zur richtigen Variable gehört.

2. Variable und Tabellenbit zusammen betrachten

```
for name, bit in zip(["A", "B", "C"], "101"):
    print(name, bit)
# A 1
# B 0
# C 1
```

zip verbindet die Einträge positionsweise. bit ist hier Text: Vergleiche mit "0" oder "1", nicht mit der Zahl 0 oder 1.

3. Literale als Text sammeln und verbinden

Für den Minterm der Zeile 101 sammeln wir ["A", "!B", "C"].

```
literals = ["A", "!B", "C"]
term = " * ".join(literals)
# Ergebnis: "A * !B * C"
```

join setzt den angegebenen Trenntext zwischen die Listenelemente. Es rechnet nichts aus. Für einen Maxterm benutzt du die umgekehrte Negationsregel, " + " als Trenntext und Klammern um den Term.

!A und A + B sind im erzeugten String unsere Logiknotation. In echtem Python heißen die Operationen not, and und or. Den Ausgabertext nicht direkt als Python-Code ausführen.

Buchbezug: Logikalgorithmus nach DDCA 2.2; Python-Erklärung als eigene Umsetzungshilfe. Eigene Erklärungen und Beispiele.

08 / SELBSTSTÄNDIG UMSETZEN

Algorithmus, Sonderfälle und Lernziel

Das Rezept für dein Tool

1. Prüfe: mindestens eine Variable, genau 2^n Ausgangswerte und ausschließlich 0 oder 1.
2. Gehe mit `enumerate(outputs)` durch die Liste. Du erhältst Zeilenindex und Ausgangswert.
3. Erzeuge aus dem Index genau n Tabellenbits.
4. Für SOP: Nur bei Ausgang 1 einen Minterm bilden; alle Minterme mit " + " verbinden.
5. Für POS: Nur bei Ausgang 0 einen Maxterm bilden; alle Maxterme mit " * " verbinden.

Was passiert bei konstanten Funktionen?

Ausgangsliste	Bedeutung	Tool-Ausgabe für SOP und POS
nur Nullen	Ausgang ist immer 0	"0"
nur Einsen	Ausgang ist immer 1	"1"

Für diese beiden Fälle vereinbaren wir bewusst eine kurze Sonderausgabe. Bei allen nichtkonstanten Funktionen soll dein Tool die **kanonischen**, ungekürzten Terme zurückgeben. Ein leerer String wäre keine brauchbare Beschreibung der konstanten Funktion.

De Morgan: die kleine Wiederholung

$$\overline{P + Q} = \overline{P} \cdot \overline{Q} \qquad \overline{P \cdot Q} = \overline{P} + \overline{Q}$$

Negation nach innen ziehen bedeutet: Verknüpfung wechseln und **beide vollständigen Teile** negieren. P und Q dürfen selbst Klammerausdrücke sein. Eine doppelte Negation hebt sich auf.

Woran erkennst du, dass der Tag geschafft ist?

Du kannst eine Zeile als Minterm und Maxterm schreiben und die unterschiedliche Negation durch Einsetzen erklären. Du kannst SOP und POS aus einer Tabelle gewinnen. Dein Tool verarbeitet XOR3 und Majority3; für Majority3 erzeugt die SOP genau vier Minterme. Du prüfst beide Ausgaben gegen alle acht Tabellenzeilen und hältst das Ergebnis in `auswertung.md` fest.

Literatur und Abgrenzung

Harris & Harris, *Digital Design and Computer Architecture, RISC-V Edition*, 2022. Kernlektüre: 2.1–2.3, Buchseiten 53–65 (PDF-Seiten 77–89). Besonders: 2.2.2 SOP, S. 56–58; 2.2.3 POS, S. 58; De Morgan und Wahrheitstabellenprüfung, S. 61–62. Ergänzend Gatterumsetzung: 2.4, S. 65.

Grundlage ist der aktuelle Kalender: W02 T1, Lerntag 7. Karnaugh-Diagramme gehören zum nächsten Lerntag. Die Beispiele und Zeichnungen hier sind eigenständig erstellt.